

Part 1: Dataset + Audio Processing

1.1 The Dataset

Base Dataset — GTZAN

- Curated by MARSYAS (an open-source audio processing framework); it is the most widely used benchmark dataset for music genre classification.
- Composition: 10 genres × 100 clips each = 1,000 audio clips total.
- Format: ~30 seconds per clip, 22,050 Hz sample rate, single channel (mono), 16-bit encoding, .wav format.

Why Noise Matters

Most existing research trains and evaluates on clean GTZAN, which does not reflect real-world listening conditions (background noise, interference, etc.). To make the model robust, a custom Noisy GTZAN dataset was built.

Building Noisy GTZAN

- Noise samples were collected from open-source projects across six categories: wind, mechanical noise, electrical feedback, white noise, factory noise, and machine-tool noise.
- Each original GTZAN clip was overlaid with these noise samples to simulate real-world conditions.
- Resulting dataset size: ~390–400 clips per genre (up from 100), since each clean clip generates multiple noisy variants.
- Specs changed from the original: still 22,050 Hz, but bumped to 32-bit encoding (from 16-bit) and clip length reduced to 10 seconds (from 30 seconds) to keep storage manageable.

Property	GTZAN (clean)	Noisy GTZAN
Clips/genre	100	~390-400
Clip length	30 sec	10 sec
Bit depth	16-bit	32-bit
Sample rate	22,050 Hz	22,050 Hz
Noise types	None	Wind, mechanical, electrical feedback, white noise, factory, machine-tool

1.2 Audio Processing — Two Parallel Tracks

The project processes audio in two distinct ways depending on downstream use:

- Track A: Hand-crafted feature extraction (tabular features) → feeds classical ML models (SVM, XGBoost, etc.)
- Track B: Mathematical signal transforms → produces a Mel-spectrogram image → feeds the deep learning model

Track A: Feature Extraction (used with GTZAN, for classical ML)

All features below are computed with librosa, generally producing a per-frame array from which mean and variance are taken to compress it into 2 summary features per descriptor.

Feature	What it captures	Function used	Output
Chromagram	Melodic/harmonic content across 12 pitch classes	librosa.feature.chroma_stft()	12-dim vector -> mean & variance -> 2 features
RMS Energy	Loudness of the signal	librosa.feature.rms()	mean & variance -> 2 features
Spectral Centroid	“Center of mass” of the frequency spectrum — where energy is concentrated	librosa.feature.spectral_centroid()	mean & variance -> 2 features
Spectral Bandwidth	Spread of frequencies around the centroid (order-p weighted std dev)	librosa.feature.spectral_bandwidth()	mean & variance -> 2 features
Spectral Rolloff	Frequency below which a set % (e.g. 85%) of spectral energy lies — describes signal shape	librosa.feature.spectral_rolloff()	mean & variance -> 2 features
Zero Crossing Rate	Smoothness/noisiness — how often the signal crosses zero amplitude	librosa.zero_crossings()	mean & variance -> 2 features
Harmony	Harmonic component of the audio	librosa.effects.harmonic()	mean & variance -> 2 features
Percussion	Percussive component of the audio	librosa.effects.percussive()	mean & variance -> 2 features
Tempo	Pace of the track, in BPM	librosa.beat.tempo()	mean tempo across frames

MFCCs	Compact representation of the audio's overall spectral envelope	librosa.feature.mfcc()	20 vectors x (mean+variance) = 40 features
-------	---	------------------------	--

MFCC pipeline in detail (since it is the richest feature)

- Audio is split into overlapping frames.
- FFT is applied to each frame → magnitude spectrum.
- Log of the magnitude is taken.
- A cosine transform (DCT) is applied to the log-spectrum → result lives in the “quefrency” domain, called the cepstrum.
- The coefficients of this cepstrum are the MFCCs.

Together, these hand-crafted descriptors form the tabular feature set later fed into Naïve Bayes, KNN, Logistic Regression, Decision Trees, Random Forest, SGD, SVM, MLP, and XGBoost (covered in Part 3).

Track B: Mathematical Preprocessing (used with Noisy GTZAN, for the deep learning model)

This track converts raw waveform into an image (Mel-spectrogram) so a CNN can treat audio classification as an image classification problem.

1. Discrete Fourier Transform (DFT)

Converts a time-domain signal into its frequency-domain (sine/cosine) components. It can be expressed as a matrix multiplication using powers of $\omega_n = e^{(-2\pi i/n)}$. Downside: $O(n^2)$ complexity — too slow for large audio arrays.

2. Fast Fourier Transform (FFT)

A computationally efficient reformulation of DFT, reducing complexity to $O(n \log n)$. For a 10-second clip at 44 kHz ($n \approx 4.4 \times 10^5$), DFT would need $\sim 10^{11}$ multiplications versus only $\sim 10^6$ for FFT — a difference of five orders of magnitude. Works natively when n is a power of 2 (recursive splitting into even/odd components); otherwise handled via zero-padding.

3. Short-Term Fourier Transform (STFT)

Applies FFT over sliding windows of the signal rather than the whole clip at once:

- Select a time window, treat it as a self-contained signal (with attenuated edges).
- Run FFT on that window.
- Compute the power spectrum.
- Represent it as a colored bar (frequency = height, power = color).

Overlapping windows are used to compensate for edge attenuation. Stacking these bars over time produces a spectrogram.

4. Spectrogram

A 2D (or 3D) representation showing signal strength across time and frequency, built by dividing audio into equal segments, applying FFT to each, and combining results. Frame size controls resolution — too few frames per unit time means information gaps.

5. Mel-Scaled Spectrogram

Human pitch perception is logarithmic, not linear (e.g., a 202 Hz gap sounds huge at low frequencies but negligible at high frequencies). The Mel scale corrects for this using an empirically determined formula relating Mel value (m) to frequency in Hz (f):

$$m = 2595 \cdot \log(1 + f/500) \quad f = 700 \cdot (10^{(m/2595)} - 1)$$

Algorithm to generate the Mel-spectrogram

- Divide audio into segments
- Extract STFT
- Convert amplitude to decibels
- Convert frequency axis to Mel scale (via a Mel filter bank)
- Combine into the final Mel-spectrogram

This Mel-spectrogram (as an image) is the actual input to the deep learning model discussed in Part 3/4 — it achieved 96.66% test accuracy versus 90.29% for the best classical ML model (XGBoost) on hand-crafted features.